

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
02.03.01 Математика и компьютерные науки

Отчёт по лабораторным работам №1–5

по дисциплине

Теория графов

Разработка программы для генерации графов и
реализации базовых алгоритмов теории графов

Вариант 17

Выполнил студент группы 5130201/40003 _____ Джабраилов А. В.

Проверил _____ Востров А. В.

«_____» _____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Постановка задачи	4
2 Математическое описание	7
2.1 Генерация графа и базовый анализ	7
2.2 Обход в ширину и кратчайшие пути	11
2.3 Сеть и потоки	13
2.4 Остовные деревья, код Прюфера и паросочетания	14
2.5 Эйлеровы графы и циклы	15
3 Особенности реализации лабораторных работ	18
3.1 Лабораторная работа №1	18
3.2 Лабораторная работа №2	20
3.3 Лабораторная работа №3	21
3.4 Лабораторная работа №4	22
3.5 Лабораторная работа №5	23
4 Результаты работы программы	25
4.1 Результаты лабораторной работы №1	25
4.2 Результаты лабораторной работы №2	31
4.3 Результаты лабораторной работы №3	32
4.4 Результаты лабораторной работы №4	33
4.5 Результаты лабораторной работы №5	37
Заключение	42
4.6 Преимущества программы	45
4.7 Недостатки программы	45
4.8 Возможности масштабирования	45
Список литературы	47

Введение

Теория графов является разделом дискретной математики, изучающим структуры, состоящие из вершин и рёбер, и отношения между ними. Графовые модели широко применяются для описания самых разных объектов и процессов: транспортных и компьютерных сетей, маршрутов, потоков, зависимостей, социальных связей и структур данных. Практическая ценность теории графов связана с тем, что многие реальные задачи естественным образом сводятся к задачам на графах, таким как поиск путей, анализ связности, построение остовов, исследование циклов, потоков и паросочетаний. Именно поэтому методы теории графов занимают важное место как в математике, так и в информатике. Также теория графов является основой нейронных сетей и искусственного интеллекта.

В рамках цикла лабораторных работ по материалам секции «Телематика» [5] была разработана единая консольная программа на языке C++. Программа позволяет последовательно выполнять генерацию графа, анализ его метрических характеристик, поиск путей, работу с потоками, построение остовов, кодирование деревьев, поиск паросочетаний, а также анализ эйлеровых циклов и базиса циклов.

Практическая ценность такого подхода состоит в том, что результаты предыдущих лабораторных работ не теряются, а переиспользуются в последующих. Например, граф, построенный в первой лабораторной работе, используется при обходе в ширину и поиске кратчайших путей во второй, преобразуется в сеть потоков в третьей, становится основой для построения минимального остова в четвёртой и служит исходными данными для эйлеризации и анализа циклической структуры в пятой. Таким образом, при выполнении каждой лабораторной работы на самом деле происходила модификация уже реализованного кода.

1. Постановка задачи

Целью работы являлась реализация набора алгоритмов теории графов и объединение их в единый программный комплекс с текстовым пользовательским интерфейсом. В ходе выполнения лабораторных работ требовалось решить следующие задачи:

Лабораторная работа №1

1. сформировать случайным образом связный ациклический ориентированный граф в соответствии с биномиальным распределением по степеням вершин;
2. реализовать использование ориентированного или неориентированного графа, полученного из сгенерированного ориентированного графа, в зависимости от дальнейшей задачи;
3. посчитать эксцентриситеты вершин;
4. выделить центр графа;
5. определить диаметральные вершины графа;
6. реализовать метод Шимбелла для сгенерированной весовой матрицы при заданном пользователем количестве рёбер;
7. реализовать генерацию весовой матрицы в соответствии с биномиальным распределением с поддержкой только положительных, только отрицательных и смешанных значений по выбору пользователя;
8. выводить минимальный и/или максимальный путь методом Шимбелла в виде матрицы;
9. определить возможность построения маршрута от одной заданной вершины до другой;
10. определять количество маршрутов между двумя вершинами;
11. реализовать пользовательское меню программы.

Лабораторная работа №2

12. реализовать обход вершин графа поиском в ширину;
13. сгенерировать весовую матрицу с положительными или отрицательными весами по выбору пользователя;
14. найти кратчайший путь между двумя выбранными пользователем вершинами классическим алгоритмом Дейкстры;
15. выводить не только расстояние, но и сам путь в виде последовательности вершин;
16. сравнить скорости работы реализованных алгоритмов по количеству итераций.

Лабораторная работа №3

17. на основе сгенерированного ориентированного графа случайным образом получить матрицу пропускных способностей;
18. на основе сгенерированного ориентированного графа случайным образом получить матрицу стоимости;
19. найти максимальный поток для полученного графа по алгоритму Форда–Фалкерсона;
20. вычислить поток минимальной стоимости величины $[2/3 \cdot max]$, где max — найденный максимальный поток;
21. использовать для задачи потока минимальной стоимости ранее реализованные алгоритмы поиска кратчайших путей.

Лабораторная работа №4

22. найти число остовных деревьев заданного неориентированного графа, используя матричную теорему Кирхгофа;
23. построить минимальный по весу остов для сгенерированного неориентированного взвешенного графа, используя алгоритм Краскала;
24. закодировать полученный остов с помощью кода Прюфера;

25. декодировать остов по коду Прюфера;
26. сохранять веса рёбер при кодировании и декодировании;
27. реализовать алгоритм нахождения максимального независимого множества рёбер на исходном графе;
28. реализовать алгоритм нахождения максимального независимого множества рёбер на полученном остове;
29. обеспечить выбор пользователем графа, на котором запускается алгоритм паросочетания.

Лабораторная работа №5

30. проверить, является ли заданный неориентированный граф эйлеровым;
31. модифицировать граф при необходимости с логированием внесённых изменений;
32. построить эйлеров цикл;
33. построить кратчайший остов для заданного неориентированного графа алгоритмом из четвёртой лабораторной работы;
34. получить фундаментальную систему циклов на основе построенного остова;
35. вывести фундаментальную систему циклов на экран;
36. получать остальные циклы на основе фундаментальных с помощью операции симметрической разности;
37. обеспечить выбор пользователем циклов, участвующих в операции симметрической разности.

Помимо собственно алгоритмов, требовалось реализовать пользовательское меню, проверки входных данных и повторное использование уже построенных структур в рамках одного сеанса работы программы.

2. Математическое описание

2.1. Генерация графа и базовый анализ

Граф определяется как пара множеств $G = (V, E)$, где V — множество вершин, а E — множество рёбер. Если рёбра задаются упорядоченными парами, граф является ориентированным; если неупорядоченными — неориентированным. Степенью вершины $d(v)$ называется число инцидентных ей рёбер. Для неориентированного графа справедлива лемма о рукопожатиях: сумма степеней всех вершин равна удвоенному числу рёбер [6].

В первой лабораторной работе используется случайная генерация графа и весов на основе биномиального распределения. Согласно справочнику Р. Н. Вадзинского [4], биномиальная случайная величина X с параметрами n и p показывает, сколько раз из n независимых испытаний произошло нужное событие, если вероятность этого события в одном испытании равна p , а вероятность противоположного исхода равна $q = 1 - p$.

Ряд распределения биномиальной случайной величины имеет вид

$$p(x) = \mathbb{P}(X = x) = C_n^x p^x q^{n-x}, \quad x = 0, 1, \dots, n,$$

где $n \geq 1$, $0 < p < 1$, $q = 1 - p$. Функция распределения задаётся кусочно:

$$F(x) = \begin{cases} 0, & x \leq 0, \\ \sum_{i=0}^k C_n^i p^i q^{n-i}, & k < x \leq k+1, \quad k = 0, 1, \dots, n-1, \\ 1, & x > n. \end{cases}$$

Производящая функция для этого распределения равна

$$\varphi_X(t) = (q + pt)^n,$$

а характеристическая функция имеет вид

$$\chi_X(t) = (q + pe^{it})^n.$$

Основные числовые характеристики биномиального распределения в

справочнике приводятся в следующем виде:

$$\mathbb{E}X = np, \quad \mathbb{D}X = npq, \quad \sigma_X = \sqrt{npq}.$$

Мода распределения определяется по формуле

$$\hat{x} = \begin{cases} \lfloor (n+1)p \rfloor, & (n+1)p \text{ не целое,} \\ (n+1)p - 1 \text{ и } (n+1)p, & (n+1)p \text{ целое.} \end{cases}$$

Коэффициент вариации, асимметрия и эксцесс соответственно равны

$$v_X = \sqrt{\frac{q}{np}}, \quad Sk = \frac{q-p}{\sqrt{npq}}, \quad Ex = \frac{1-6pq}{npq}.$$

Ниже приведены три варианта представления биномиального распределения. На рис. 1 показан исходный вид распределения из справочника Р. Н. Вадзинского для случая $n = 4$ и нескольких значений параметра p . На рис. 2 приведена построенная по 1000 сгенерированным значениям гистограмма для тех же исходных параметров $n = 4$, что позволяет сопоставить теоретическую форму распределения и результат моделирования. На рис. 3 показана гистограмма распределения, которое уже используется непосредственно в программе.

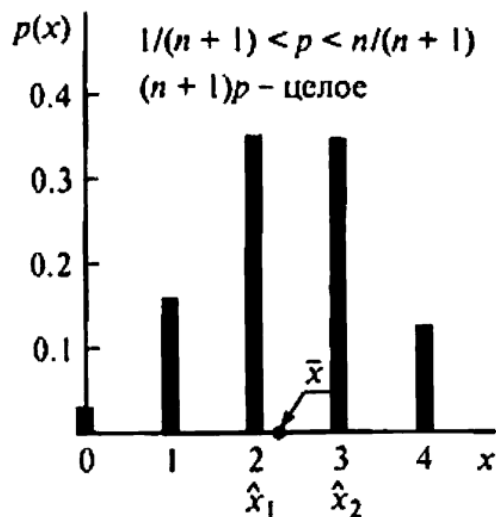


Рис. 1: Исходный вид биномиального распределения из справочника Р. Н. Вадзинского



Рис. 2: Гистограмма 1000 сгенерированных значений биномиального распределения для исходных параметров $n = 4$

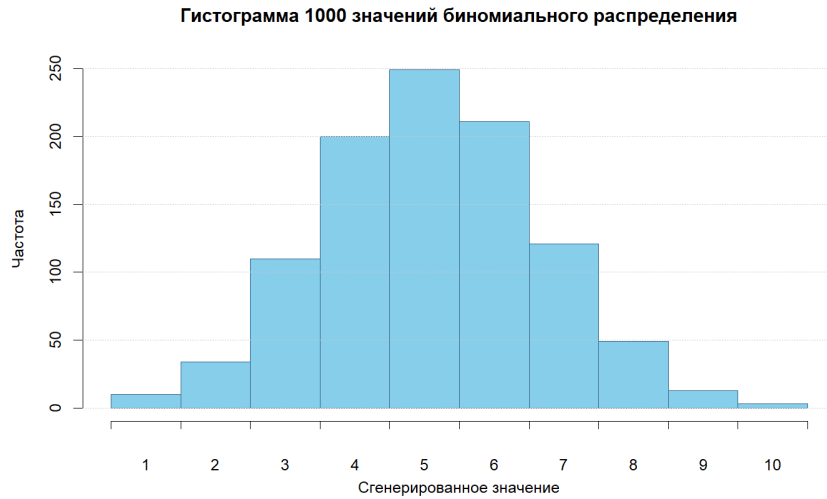


Рис. 3: Гистограмма 1000 сгенерированных значений биномиального распределения, используемого в программе

В справочнике иллюстрации построены для распределения с параметром $n = 4$, поэтому случайная величина принимает значения только от 0 до 4, а среднее значение равно np . В программе параметры распределения были изменены на $n = 10$ и $p = 0.5$. Это увеличивает диапазон возможных значений до отрезка $0 \dots 10$ и даёт математическое ожидание $\mathbb{E}X = np = 5$ вместо среднего значения 2 для случая $n = 4, p = 0.5$. В результате при генерации графа в среднем получаются большие степени вершин, а значит и большее число рёбер, что делает графы более насыщенными и удобными для демонстрации алгоритмов.

В программе генерация биномиальной случайной величины выполняется стандартным имитационным способом для дискретных распределений. Сначала вычисляется вероятность первого значения

$$p(0) = q^n,$$

после чего следующие вероятности пересчитываются рекуррентно:

$$p(x) = p(x-1) \cdot \frac{p}{q} \cdot \frac{n+1-x}{x}, \quad x = 1, 2, \dots, n.$$

Далее генерируется равномерное случайное число $r \in [0, 1]$, из него последовательно вычитаются значения $p(0), p(1), p(2), \dots$, и первым подходящим результатом считается то значение x , на котором остаток становится отрица-

тельным. Такой алгоритм эквивалентен генерации по функции распределения и соответствует стандартному способу имитационного моделирования дискретных случайных величин.

Блок-схема стандартного алгоритма генерации биномиально распределённой случайной величины, приведённая в справочнике Р. Н. Вадзинского, показана на рис. 4.

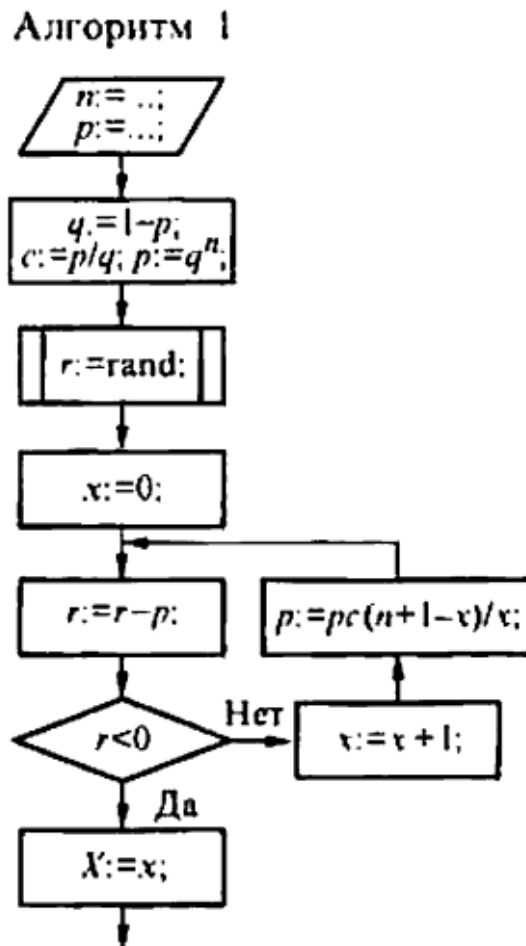


Рис. 4: Блок-схема алгоритма генерации биномиально распределённой случайной величины по справочнику Р. Н. Вадзинского

В рамках работы биномиальное распределение используется при генерации степеней вершин, при генерации весов рёбер и при генерации параметров сети потоков. Для весов дополнительно вводится пользовательский выбор знака: только положительные значения, только отрицательные значения или смешанный режим. При этом нулевой по модулю вес исключается, чтобы каждое ребро имело ненулевой вес.

Для связного графа расстоянием $d(u, v)$ называется длина кратчайшей цепи между вершинами u и v , измеряемая числом рёбер. Эксцентриситет вер-

шины определяется формулой

$$e(v) = \max_{u \in V} d(v, u),$$

радиус графа равен

$$R(G) = \min_{v \in V} e(v),$$

а диаметр — максимальному эксцентриситету по всем вершинам. Вершины с эксцентриситетом, равным радиусу, образуют центр графа, а вершины с максимальным эксцентриситетом являются диаметральными.

Метод Шимбелла использует степени матрицы смежности для анализа маршрутов фиксированной длины. В данной работе эта идея обобщается на взвешенный случай: для путей из k рёбер вычисляются минимальные и максимальные суммарные веса. Поиск маршрутов между двумя вершинами сводится к перебору всех простых путей между заданными концами.

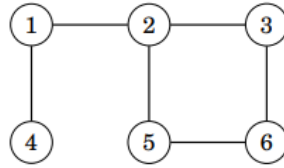
2.2. Обход в ширину и кратчайшие пути

Поиск в ширину, или BFS, выполняет обход графа слоями относительно стартовой вершины. Сначала посещается исходная вершина, затем все вершины на расстоянии одного ребра, потом на расстоянии двух рёбер и так далее. Такой порядок гарантирует нахождение кратчайших путей по числу рёбер в невзвешенном графе.

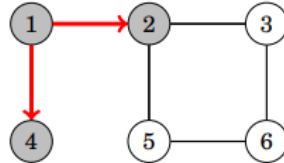
Алгоритм поиска в ширину можно визуализировать следующим образом (рис. 5) [2].

Example

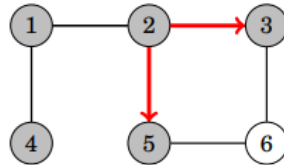
Let us consider how breadth-first search processes the following graph:



Suppose that the search begins at node 1. First, we process all nodes that can be reached from node 1 using a single edge:



After this, we proceed to nodes 3 and 5:



Finally, we visit node 6:

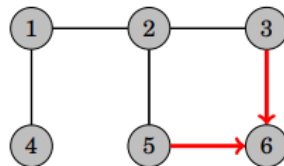


Рис. 5: Визуализация алгоритма поиска в ширину.

Алгоритм Дейкстры предназначен для поиска кратчайших путей во взвешенном графе с неотрицательными весами. Каждой вершине сопоставляется текущая оценка расстояния от источника. На каждом шаге выбирается непосещённая вершина с минимальной меткой, после чего выполняется релаксация исходящих рёбер. После завершения работы алгоритма метка целевой вершины равна длине кратчайшего пути, а массив предшественников позволяет восстановить сам путь.

Алгоритм Дейкстры можно визуализировать следующим образом (рис. 6) [5].

Вход: орграф $G(V, E)$, заданный матрицей длин дуг $C : \text{array } [1..p, 1..p] \text{ of real}$; s и t — вершины графа.

Выход: векторы $T : \text{array } [1..p] \text{ of real}$; и $H : \text{array } [1..p] \text{ of } 0..p$. Если вершина v лежит на кратчайшем пути от s к t , то $T[v]$ — длина кратчайшего пути от s к v ; $H[v]$ — вершина, непосредственно предшествующая v на кратчайшем пути.

```

for  $v$  from 1 to  $p$  do
   $T[v] := \infty$  { кратчайший путь неизвестен }
   $X[v] := 0$  { все вершины не отмечены }
end for
 $H[s] := 0$  {  $s$  ничего не предшествует }
 $T[s] := 0$  { кратчайший путь имеет длину 0... }
 $X[s] := 1$  { ... и он известен }
 $v := s$  { текущая вершина }
 $M := \{\}$  { обновление пометок }
for  $u \in \Gamma(v)$  do
  if  $X[u] = 0$  &  $T[u] > T[v] + C[v, u]$  then
     $T[u] := T[v] + C[v, u]$  { найден более короткий путь из  $s$  в  $u$  через  $v$  }
     $H[u] := v$  { запоминаем его }
  end if
end for
 $m := \infty$ ;  $v := 0$ 
{ поиск конца кратчайшего пути }
for  $u$  from 1 to  $p$  do
  if  $X[u] = 0$  &  $T[u] < m$  then
     $v := u$ ;  $m := T[u]$  { вершина  $v$  заканчивает кратчайший путь из  $s$  }
  end if
end for
if  $v = 0$  then
  stop { нет пути из  $s$  в  $t$  }
end if
if  $v = t$  then
  stop { найден кратчайший путь из  $s$  в  $t$  }
end if
 $X[v] := 1$  { найден кратчайший путь из  $s$  в  $v$  }
goto  $M$ 

```

Рис. 6: Алгоритм Дейкстры.

Во второй лабораторной работе дополнительно сравниваются трудоёмкости BFS и алгоритма Дейкстры по числу итераций и релаксаций на одном и том же графе. Было выяснено, что Дейкстра менее эффективен, чем BFS.

2.3. Сеть и потоки

Сетью называется ориентированный граф $N = (V, E)$ с выделенными вершинами s (источник) и t (сток), где каждой дуге $e \in E$ сопоставлены пропускная способность $c(u, v)$ и стоимость $w(u, v)$.

Число $w(f) = \text{div}(f, s)$ называется величиной потока f . Если $f(u, v) = c(u, v)$, то дуга (u, v) называется насыщенной.

Стоимость потока определяется выражением

$$\text{cost}(f) = \sum_{(u,v) \in E} f(u, v) w(u, v).$$

Задача о потоке минимальной стоимости фиксированной величины состоит в поиске допустимого потока заданного значения с минимальной суммарной стоимостью. В лабораторной работе №4 целевая величина берётся равной $\lfloor \frac{2}{3} \maxFlow \rfloor$.

2.4. Остовные деревья, код Прюфера и паросочетания

Остовным деревом связного графа называется подграф, содержащий все вершины и не содержащий циклов.

Теорема Кирхгофа. Число остовных деревьев равно алгебраическому дополнению любого элемента матрицы Кирхгофа. Матрица Кирхгофа задаётся формулой

$$L = D - A,$$

где D — диагональная матрица степеней, а A — матрица смежности.

Минимальным остовным деревом называется остов, имеющий минимальную сумму весов. Алгоритм Краскала строит такой остов, просматривая рёбра в порядке неубывания веса и добавляя очередное ребро только в том случае, если оно не образует цикл.

Алгоритм краскала описан на рис. 7 [5].

```

Вход: список  $E$  рёбер графа  $G$  с длинами, упорядоченный в порядке возрастания
длин.
Выход: множество  $T$  рёбер кратчайшего остова.
 $T := \emptyset$ 
 $k := 1$  { номер рассматриваемого ребра }
for  $i$  from 1 to  $p - 1$  do
  while  $z(T + E[k]) > 0$  do
     $k := k + 1$  { пропустить это ребро }
  end while
   $T := T + E[k]$  { добавить это ребро в  $SST$  }
   $k := k + 1$  { и исключить его из рассмотрения }
end for

```

Рис. 7: Алгоритм Краскала.

Код Прюфера задаёт взаимно однозначное соответствие между помеченными деревьями на p вершинах и последовательностями длины $p - 2$. При кодировании последовательно удаляется лист с минимальным номером, а в последовательность записывается его сосед. Независимое множество рёбер в графе является паросочетанием. В работе рассматриваются как жадное

максимальное по включению паросочетание, так и максимальное по мощности паросочетание.

Кодирование и декодирование кода Прюфера изображены на рис. ?? и рис. ?? [5].

Вход: Нетривиальное упорядоченное дерево T , заданное списками смежности
 Γ : **array** [1.. p] of $\uparrow N$, где $N = \text{record } v: 1..p; n: \uparrow N \text{ end record}$, причём узлы перенумерованы в прямом порядке.
Выход: Массив C : **array** [1.. $2q$] of 0..1, являющийся кодом дерева T .
 $i := 0$ { количество заполненных разрядов кода }
 $\text{TraverseTree}(1)$ { корневой узел имеет номер 1 }

Вход: Число n : 1.. p — номер текущего узла.
Выход: Заполнение двух разрядов кода C .
 $i := i + 1$; $C[i] := 1$ { отмечаем вход в узел }
 $d := \Gamma[n]$ { указатель на первый элемент списка сыновей }
while $d \neq \text{nil}$ **do**
 $\text{TraverseTree}(d.v)$ { построение кода поддерева очередного сына }
 $d := d.n$ { выбор следующего сына }
end while
 $i := i + 1$; $C[i] := 0$ { отмечаем выход из узла }

Рис. 8: Алгоритм кодирования кода Прюфера.

Вход: Массив C : **array** [1.. $2q$] of 0..1 — код упорядоченного дерева T .
Выход: Массив Γ : **array** [1.. p] of $\uparrow N$, где $N = \text{record } v: 1..p; n: \uparrow N \text{ end record}$ — списки смежности упорядоченного дерева T .
 $p := 1$ { счётчик узлов }
 $\Gamma[1] := \text{nil}$ { корень }
 $n := 1$ { текущий узел }
for i **from** 1 **to** $2q$ **do**
 if $C[i] = 1$ **then**
 $p := p + 1$ { номер нового узла }
 $\Gamma[p] := \text{nil}$ { новый узел пока листовой }
 $d := \text{NewNode}(p, \text{nil})$ { дуга от текущего узла к новому узлу }
 $\text{Append}(\Gamma[n], d)$ { добавить узел в список смежности }
 $n \rightarrow S$ { положить номер текущего узла на стек }
 $n := p$ { перейти в новый узел }
 else
 $n \leftarrow S$ { снять со стека и перейти в новый узел }
 end if
end for

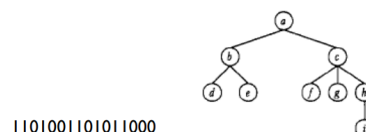


Рис. 9: Алгоритм декодирования кода Прюфера.

2.5. Эйлеровы графы и циклы

Эйлеровым циклом называется цикл, проходящий по каждому ребру графа ровно один раз. Граф является эйлеровым тогда и только тогда, когда он связан и степени всех его вершин чётны. Если связный граф имеет ровно две вершины нечётной степени, то он является полуэйлеровым и допускает эйлерову цепь, но не цикл.

Пусть $T(V, E_T)$ — остов связного графа $G(V, E)$. Каждая хорда $e \in E \setminus E_T$ вместе с единственным путём в остове между её концами образует фундаментальный цикл. Все такие циклы образуют фундаментальную систему циклов. Её мощность равна цикломатическому числу

$$m(G) = |E| - |V| + 1$$

для связного графа.

Операцией сложения в пространстве циклов служит симметрическая разность множеств рёбер. В результате симметрической разности фундаментальных циклов можно получить как один цикл, так и объединение нескольких циклов, то есть чётный подграф, который затем раскладывается на простые циклы.

С точки зрения пространства подмножеств рёбер каждый цикл можно рассматривать как вектор над двоичной арифметикой: ребро входит в подмножество, если соответствующий коэффициент равен 1, и не входит, если коэффициент равен 0. При такой интерпретации сложению векторов соответствует симметрическая разность множеств рёбер. Совокупность всех симметрических разностей фундаментальных циклов содержит не только все циклы графа, но и некоторые объединения циклов; такие объекты в лекционных материалах называются циклическими векторами.

Фундаментальная система циклов является базисом подпространства циклов. Фундаментальные циклы независимы, поскольку каждый из них содержит индивидуальную хорду остова, а циклический ранг $m(G)$ равен размерности этого подпространства.

****Алгоритм Иерихольцера****[2] — это эффективный метод построения эйлерова цикла. Алгоритм состоит из нескольких этапов, на каждом из которых в цикл добавляются новые рёбра. Разумеется, предполагается, что граф содержит эйлеров цикл; в противном случае алгоритм Иерихольцера не сможет его найти.

Сначала алгоритм строит цикл, содержащий часть рёбер графа (не обязательно все рёбра). Затем этот цикл постепенно расширяется путём присоединения к нему подциклов. Процесс продолжается до тех пор, пока в цикл не будут включены все рёбра графа.

Для расширения текущего цикла алгоритм каждый раз находит вершину (x), которая уже принадлежит циклу, но из которой выходит хотя бы одно ребро, ещё не включённое в цикл. Из вершины (x) строится новый путь, состоящий только из ещё не использованных рёбер. Рано или поздно этот путь вернётся в вершину (x), образуя новый подцикл.

Затем полученный подцикл встраивается в уже существующий цикл. Повторяя этот процесс, алгоритм постепенно включает в итоговый цикл все рёбра графа.

Если граф содержит только эйлеров путь, алгоритм Иерихольцера всё равно можно использовать. Для этого в граф временно добавляют дополнительное ребро, после чего строят эйлеров цикл. Затем добавленное ребро удаляют из полученного цикла, и оставшаяся последовательность рёбер образует эйлеров путь.

Например, в неориентированном графе, содержащем ровно две вершины нечётной степени, дополнительное ребро добавляется между этими вершинами. После построения эйлерова цикла это ребро удаляется, и полученная последовательность рёбер представляет собой искомый эйлеров путь.

3. Особенности реализации лабораторных работ

3.1. Лабораторная работа №1

Основной модуль первой лабораторной работы реализован в классе `AcyclicGraphBuilder`. Он отвечает за генерацию степеней и весов по биномиальному закону, построение графа и повторную генерацию в случае несвязности. Для ориентированного режима рёбра добавляются только при $u < v$, что исключает появление ориентированных циклов.

Метод `sampleBinomial(n, p)`

Вход: параметры биномиального распределения n и p , а также внутреннее состояние генератора случайных чисел класса.

Выход: одно целое случайное значение, распределённое по биномиальному закону.

Описание: генерирует случайное число по биномиальному распределению. Код представлен в листинге 1.

```
double u = dist(m_rng);
double cumulative = 0.0;
for (int k = 0; k <= n; ++k) {
    cumulative += std::exp(logCombination(n, k) + k * std::log(p)
        + (n - k) * std::log(1.0 - p));
    if (u <= cumulative) return k;
...
}
```

Листинг 1: Фрагмент функции `sampleBinomial(n, p)`

Метод `generateAcyclicGraph(vertices, directed, weightSign)`

Вход: количество вершин, признак ориентированности графа, режим генерации весов, а также внутренние параметры биномиального распределения, закреплённые в классе.

Выход: указатель на сгенерированный граф, содержащий вершины, рёбра и веса.

Описание: строит граф с заданным числом вершин и режимом генерации весов.

Код представлен в листинге 2.

```

auto graph = std::make_unique<AdjacencyGraph>(directed);
for (int i = 1; i <= vertexCount; ++i) {
    graph->addVertex(i);
}
for (const auto& [u, v] : selectedEdges) graph->addEdge(u, v,
    generateWeight(weightSign));
...

```

Листинг 2: Фрагмент функции `generateAcyclicGraph(vertices, directed, weightSign)`

Вычисление эксцентриситетов реализовано в классе `GraphAnalyzer`. Для каждой вершины выполняется BFS, после чего по таблице расстояний вычисляются радиус, диаметр, центр и диаметральные вершины. Метод Шимбелла реализован в классе `KPathCalculator`, а поиск всех простых путей между двумя вершинами вынесен в класс `SimplePathFinder`.

Метод `analyze()`

Вход: граф, переданный объекту при создании класса.

Выход: структура `EccentricityData`, содержащая эксцентриситеты, радиус, диаметр, центр графа, диаметральные вершины и набор диаметральных путей.

Описание: находит основные метрические характеристики графа.

Код представлен в листинге 3.

```

EccentricityData result;
auto vertices = m_graph.vertexIds();
if (vertices.empty()) return result;
auto edgeCounts = computeEdgeCountPaths();
for (int v : vertices) {
    ...
}

```

Листинг 3: Фрагмент функции `analyze()`

Метод `compute(edgeCount)`

Вход: требуемое число рёбер k и граф, связанный с объектом класса.

Выход: структура `ShimbellOutput`, содержащая матрицу минимальных весов, матрицу максимальных весов и само значение k .

Описание: вычисляет минимальные и максимальные веса путей длины k .

Код представлен в листинге 4.

```
WeightTable baseMatrix = buildWeightMatrix();
WeightTable minMatrix = baseMatrix;
WeightTable maxMatrix = baseMatrix;
for (int step = 2; step <= edgeCount; ++step) {
    minMatrix = multiplyMinPlus(minMatrix, baseMatrix);
    ...
}
```

Листинг 4: Фрагмент функции compute(edgeCount)

3.2. Лабораторная работа №2

Во второй лабораторной работе обход в ширину реализован в классе BreadthFirstSearch. Он использует очередь, множество посещённых вершин и массив расстояний от стартовой вершины. Дополнительно сохраняется статистика по числу итераций.

Метод run(start)

Вход: стартовая вершина и граф, закреплённый за объектом BreadthFirstSearch.

Выход: структура BfsResult, содержащая порядок обхода, расстояния и число итераций.

Описание: выполняет поиск в ширину от выбранной вершины.

Код представлен в листинге 5.

```
BfsResult result;
std::queue<int> q;
std::unordered_set<int> used;
q.push(start);
used.insert(start);
...
```

Листинг 5: Фрагмент функции run(start)

Для кратчайших путей используется класс DijkstraShortestPath. Он хранит расстояния, предшественников и приоритетную очередь. После завершения работы алгоритма путь восстанавливается по массиву предков, а статистика по релаксациям используется для сравнения с BFS.

Метод `compute(start, finish)`

Вход: начальная и конечная вершины, а также граф, сохранённый в объекте `DijkstraShortestPath`.

Выход: структура `ShortestPathResult`, содержащая длину кратчайшего пути, сам путь и статистику релаксаций.

Описание: находит кратчайший путь между двумя вершинами алгоритмом Дейкстры.

Код представлен в листинге 6.

```
std::priority_queue<State, std::vector<State>, std::greater<State>> pq;
distances[start] = 0.0;
pq.push({0.0, start});
while (!pq.empty()) {
    auto [dist, v] = pq.top();
    ...
}
```

Листинг 6: Фрагмент функции `compute(start, finish)`

3.3. Лабораторная работа №3

Построение сети потоков реализовано в классе `FlowNetworkBuilder`. На основе текущего ориентированного графа он генерирует матрицы пропускных способностей и стоимостей, а затем создаёт объект `FlowNetwork`. Максимальный поток вычисляется классом `MaxFlowSolver`, а поток минимальной стоимости — классом `MinCostFlowSolver`.

Метод `build()`

Вход: ориентированный граф, сохранённый в объекте `FlowNetworkBuilder`.

Выход: объект `FlowNetwork`, содержащий матрицы пропускных способностей, стоимостей и начальный нулевой поток.

Описание: строит сеть потоков по текущему ориентированному графу.

Код представлен в листинге 7.

```
FlowNetwork network(m_graph.vertexIds());
for (const auto& edge : m_graph.edges()) {
```

```

        network.setCapacity(edge.from, edge.to, generateCapacity());
        network.setCost(edge.from, edge.to, generateCost());
    }
    ...

```

Листинг 7: Фрагмент функции `build()`

Метод `compute(source, sink)`

Вход: сеть потоков, исток и сток.

Выход: структура `MaxFlowResult`, содержащая значение максимального потока, финальную матрицу потока и журнал увеличений.

Описание: находит максимальный поток.

Код представлен в листинге 8.

```

MaxFlowResult result;
while (true) {
    auto path = findAugmentingPath(source, sink);
    if (path.empty()) break;
    augment(path);
    ...
}

```

Листинг 8: Фрагмент функции `compute(source, sink)`

3.4. Лабораторная работа №4

Для четвёртой лабораторной работы используются несколько независимых модулей. Класс `KirchhoffCounter` строит матрицу Лапласа и вычисляет число остовных деревьев по определителю минора. Класс `KruskalMST` строит минимальный остов с помощью системы непересекающихся множеств. Класс `PruferEncoder` кодирует и декодирует дерево, а модули `GreedyMaximalMatching` и `EdmondsMatching` находят паросочетания.

Метод `countSpanningTrees()`

Вход: неориентированный граф, закреплённый за объектом `KirchhoffCounter`.

Выход: целое число, равное количеству остовных деревьев графа.

Описание: вычисляет число остовных деревьев по теореме Кирхгофа.

Код представлен в листинге 9.

```
auto laplacian = buildLaplacian();
laplacian.pop_back();
for (auto& row : laplacian) {
    row.pop_back();
}
...
```

Листинг 9: Фрагмент функции countSpanningTrees()

Метод buildMST()

Вход: взвешенный неориентированный граф.

Выход: структура KruskalResult, содержащая построенный остов, список выбранных рёбер, суммарный вес и признак связности исходного графа.

Описание: строит минимальный остов алгоритмом Краскала.

Код представлен в листинге 10.

```
KruskalResult result;
result.totalWeight = 0.0;
result.wasConnected = true;
result.spanningTree = std::make_unique<AdjacencyGraph>(m_graph.
    isDirected());
for (int v : m_graph.vertexIds()) {
    ...
}
```

Листинг 10: Фрагмент функции buildMST()

3.5. Лабораторная работа №5

Проверка эйлеровости и построение обхода выполняются в классе EulerianCycleBuilder. Сначала граф копируется, затем при необходимости соединяются рёберные компоненты и исправляются нечётные степени. После этого запускается алгоритм Иерихольцера ??. Для исследования цикловой структуры используется класс FundamentalCycleSystem, который строит фундаментальные циклы, выполняет симметрическую разность и разложение чётного подграфа на простые циклы.

Метод `compute(mode)`

Вход: исходный неориентированный граф, закреплённый за объектом `EulerianCycleBuilder`, и режим эйлеризации, разрешающий или запрещающий мультиграф.

Выход: структура `EulerianCycleResult`, содержащая сведения о необходимости модификаций, журнал добавленных рёбер, модифицированный граф и итоговый эйлеров обход.

Описание: проверяет граф, при необходимости эйлеризует его и строит эйлеров обход.

Код представлен в листинге 11.

```
EulerianCycleResult result;
result.success = false;
result.modifiedGraph = cloneGraph(m_graph);
AdjacencyGraph& G = *result.modifiedGraph;
if (G.isDirected() || G.edges().empty()) {
    ...
}
```

Листинг 11: Фрагмент функции `compute(mode)`

Метод `compute()`

Вход: исходный граф и остов, сохранённые в объекте `FundamentalCycleSystem`.

Выход: список всех фундаментальных циклов.

Описание: вычисляет систему фундаментальных циклов для выбранного остова.

Код представлен в листинге 12.

```
std::vector<FundamentalCycle> cycles;
const auto treeEdgeSet = collectTreeEdges(m_tree);
for (const WeightedEdge& e : m_graph.edges()) {
    const auto key = normEdge(e.from, e.to);
    if (treeEdgeSet.count(key) > 0) {
        ...
    }
}
```

Листинг 12: Фрагмент функции `compute()`

4. Результаты работы программы

После запуска программа выводит главное текстовое меню, в котором все действия сгруппированы по лабораторным работам. Пользователь может последовательно переходить между генерацией графа, анализом его свойств, алгоритмами потоков, остовов и эйлеровых обходов. Общий вид меню показан на рис. 10.

4.1. Результаты лабораторной работы №1

При выборе варианта 1 программа предлагает ввести количество вершин, выбрать, будет граф ориентированным или нет, а также указать допустимые веса: положительные, отрицательные или смешанные. Примеры генерации ориентированного и неориентированного графов показаны на рис. 11 и 12.

```

===== MAIN MENU =====

--- Lab 1 ---
1. Generate random graph
2. Calculate eccentricities & center
3. Shimbell's method (k-edge paths)
4. Find all paths between vertices
5. Show graph details
6. Compare distribution statistics
7. Show adjacency matrix
8. Show weight matrix (Shimbell k=1)

--- Lab 2 ---
9. BFS traversal
10. Dijkstra's shortest path
11. Compare BFS vs Dijkstra (speed)

--- Lab 3 ---
12. Build flow network from current directed graph
13. Show capacity matrix
14. Show cost matrix
15. Find maximum flow
16. Find minimum-cost flow for [2/3 * max]
17. Show flow network details

--- Lab 4 ---
18. Count spanning trees (Kirchhoff's theorem)
19. Build minimum spanning tree (Kruskal)
20. Show spanning tree details
21. Encode spanning tree to Prufer code
22. Decode last Prufer code back to a tree
23. Maximal matching (independent edge set)

--- Lab 5 ---
24. Build Eulerian cycle/path (modify graph if needed)
25. Show fundamental cycle system (uses MST)
26. Combine cycles via symmetric difference

--- Custom ---
1001. Toggle hiding Lab 1 in menu
1002. Toggle hiding Lab 2 in menu
1003. Toggle hiding Lab 3 in menu
1004. Toggle hiding Lab 4 in menu
1005. Toggle hiding Lab 5 in menu
1006. Toggle hiding Custom in menu
101. Export current graph to Mermaid
102. Export current flow network to Mermaid
103. Export current spanning tree to Mermaid
0. Quit
=====
> █

```

Рис. 10: Главное меню программы

```

> 1

--- Graph Generation ---
Number of vertices: 8
Graph type (1=Directed, 2=Undirected): 1

>>> Choose weight distribution:
1. Positive values only
2. Negative values only
3. Mixed (positive and negative)
> 1
[!] Warning: Could only add 25 edges (max possible for 8 vertices: 28)

[OK] Graph created successfully!
Vertices: 8
Edges: 25
Degree params: n=10, p=0.5
Weight params: n=10, p=0.5

=== BINOMIAL DISTRIBUTION PARAMETERS ===
For degrees B(10, 0.5):
Expected mean: 5
Expected var: 2.5
Expected std: 1.58114
Mode: 5

For weights B(10, 0.5):
Expected mean: 5
Expected var: 2.5

=== ACTUAL GRAPH STATISTICS ===
Vertices: 8
Edges: 25
Mean degree: 3.125
Variance: 3.60938
Std deviation: 1.89984
Min degree: 0
Max degree: 6

--- Deviation from Theory ---
Mean error: 1.875
Var error: 1.10938

```

Рис. 11: Генерация ориентированного графа

```

===== MAIN MENU =====
> 1

--- Graph Generation ---
Number of vertices: 8
Graph type (1=Directed, 2=Undirected): 2

>>> Choose weight distribution:
  1. Positive values only
  2. Negative values only
  3. Mixed (positive and negative)
> 3

[OK] Graph created successfully!
  Vertices: 8
  Edges: 19
  Degree params: n=10, p=0.5
  Weight params: n=10, p=0.5

=== BINOMIAL DISTRIBUTION PARAMETERS ===
For degrees B(10, 0.5):
  Expected mean: 5
  Expected var: 2.5
  Expected std: 1.58114
  Mode: 5

For weights B(10, 0.5):
  Expected mean: 5
  Expected var: 2.5

=== ACTUAL GRAPH STATISTICS ===
Vertices: 8
Edges: 19
Mean degree: 4.75
Variance: 0.9375
Std deviation: 0.968246
Min degree: 4
Max degree: 7

--- Deviation from Theory ---
Mean error: 0.25
Var error: 1.5625

```

Рис. 12: Генерация неориентированного графа

После генерации пользователь может вывести матрицу смежности и списки смежности текущего графа. Пример такого вывода показан на рис. 13.

```

===== MAIN MENU =====
> 2

--- Eccentricity Computation (by edge count) ---

Vertex Eccentricities (in edges):
v0: 2
v1: 1
v2: 1
v3: 1
v4: 1
v5: 1
v6: 1
v7: 0

Radius:      0 (edges)
Diameter:    2 (edges)
Center:      v7
Diametrical vertices: v0

Diametrical paths (6 total):

Between v0 and v2 (1 paths):
v0 -> v1 -> v2

Between v0 and v7 (5 paths):
v0 -> v5 -> v7
v0 -> v3 -> v7
v0 -> v1 -> v7
v0 -> v4 -> v7
v0 -> v6 -> v7

```

Рис. 13: Вывод матрицы и списков смежности графа

На следующем этапе программа вычисляет статистики степеней вершин и выводит основные числовые характеристики сгенерированного графа. Пример результата показан на рис. 14.

```

===== MAIN MENU =====
> 3

Path length k (number of edges): 3

--- Minimum Weight Paths ---
i = no path

```

	0	1	2	3	4	5	6	7
0	0	i	i	i	12	14	14	14
1	i	0	i	i	i	14	14	12
2	i	i	0	i	i	i	15	16
3	i	i	i	0	i	i	18	18
4	i	i	i	i	0	i	i	18
5	i	i	i	i	i	0	i	i
6	i	i	i	i	i	i	0	i
7	i	i	i	i	i	i	i	0

```

--- Maximum Weight Paths ---
i = no path

```

	0	1	2	3	4	5	6	7
0	0	i	i	i	17	18	18	20
1	i	0	i	i	i	19	19	21
2	i	i	0	i	i	i	15	18
3	i	i	i	0	i	i	18	21
4	i	i	i	i	0	i	i	18
5	i	i	i	i	i	0	i	i
6	i	i	i	i	i	i	0	i
7	i	i	i	i	i	i	i	0

Рис. 14: Статистики степеней вершин графа

При выборе пункта анализа эксцентриситетов программа вычисляет расстояния по числу рёбер, затем выводит эксцентриситеты, радиус, диаметр, центр и диаметральные вершины. Пример такого анализа показан на рис. 15.

```

===== MAIN MENU =====
> 4

Start vertex: 3
End vertex: 7
[OK] Found 8 path(s)
Paths:
  v3 -> v6 -> v7
  v3 -> v7
  v3 -> v5 -> v7
  v3 -> v5 -> v6 -> v7
  v3 -> v4 -> v5 -> v7
  v3 -> v4 -> v5 -> v6 -> v7
  v3 -> v4 -> v6 -> v7
  v3 -> v4 -> v7

```

Рис. 15: Вычисление эксцентриситетов, радиуса, диаметра и центра графа

Для метода Шимбелла программа предлагает задать число рёбер в искомом пути, после чего строит матрицы минимальных и максимальных весов маршрутов фиксированной длины. Результат работы метода показан на рис. 16.

```
===== MAIN MENU =====  
> 5  
  
=== GRAPH INFORMATION ===  
Type:      Directed  
Vertices:   8  
Edges:      25  
Vertex IDs: 0 1 2 3 4 5 6 7
```

Рис. 16: Результат работы метода Шимбелла

При поиске маршрутов между двумя вершинами программа просит ввести начальную и конечную вершины, затем выводит все найденные простые пути и их количество. Пример результата показан на рис. 17.

```
===== MAIN MENU =====  
> 6  
  
=== BINOMIAL DISTRIBUTION PARAMETERS ===  
For degrees B(10, 0.5):  
  Expected mean:  5  
  Expected var:   2.5  
  Expected std:   1.58114  
  Mode:           5  
  
For weights B(10, 0.5):  
  Expected mean:  5  
  Expected var:   2.5  
  
=== ACTUAL GRAPH STATISTICS ===  
Vertices:      8  
Edges:         25  
Mean degree:   3.125  
Variance:      3.60938  
Std deviation: 1.89984  
Min degree:    0  
Max degree:    6  
  
--- Deviation from Theory ---  
Mean error:    1.875  
Var error:     1.10938
```

Рис. 17: Поиск всех простых путей между двумя вершинами

4.2. Результаты лабораторной работы №2

При выборе пункта BFS программа запрашивает стартовую вершину и затем выводит порядок обхода, а также расстояния по числу рёбер до остальных вершин. Пример результата показан на рис. 18.

```
===== MAIN MENU =====
> 7

=== ADJACENCY MATRIX (direct edges) ===
Shows direct edges between vertices (1 edge)
1 = edge exists, 0 = no edge
```

	0	1	2	3	4	5	6	7
0	0	1	0	1	1	1	1	0
1	0	0	1	1	1	1	1	1
2	0	0	0	0	1	1	1	1
3	0	0	0	0	1	1	1	1
4	0	0	0	0	0	1	1	1
5	0	0	0	0	0	0	1	1
6	0	0	0	0	0	0	0	1
7	0	0	0	0	0	0	0	0

Рис. 18: Результат обхода графа поиском в ширину

Для алгоритма Дейкстры программа предлагает выбрать две вершины и затем выводит длину кратчайшего пути и сам путь в виде последовательности вершин. Пример результата приведён на рис. 19.

```
===== MAIN MENU =====
> 8

=== WEIGHT MATRIX (Shimbell, k=1) ===
Minimum weight paths with EXACTLY 1 edge
1 = no path with exactly 1 edge
```

	0	1	2	3	4	5	6	7
0	0	5	1	5	3	4	4	1
1	1	0	4	6	5	4	4	6
2	1	1	0	1	3	5	8	5
3	1	1	1	0	6	7	5	7
4	1	1	1	1	0	7	7	5
5	1	1	1	1	1	0	5	8
6	1	1	1	1	1	1	0	6
7	1	1	1	1	1	1	1	0

Рис. 19: Поиск кратчайшего пути алгоритмом Дейкстры

После этого можно выполнить сравнение алгоритмов по количеству итераций. Программа выводит показатели для BFS и алгоритма Дейкстры на одном и том же графе. Пример такого сравнения показан на рис. 20.

```
===== MAIN MENU =====
> 9

--- BFS Traversal ---
Start vertex: 2

[OK] BFS completed in 27 iterations
Maximum depth (levels): 1

--- BFS Levels ---
Level 0: [2]
Level 1: [6, 5, 7, 4]
```

Рис. 20: Сравнение алгоритмов BFS и Дейкстры

4.3. Результаты лабораторной работы №3

Для задач на потоки программа сначала строит сеть на основе текущего ориентированного графа, генерируя матрицы пропускных способностей и стоимостей. Пример построенной сети и её параметров показан на рис. 21.

```
===== MAIN MENU =====  
> 10  
  
--- Dijkstra's Shortest Path ---  
Start vertex: 3  
End vertex: 7  
  
[OK] Dijkstra completed in 42 iterations  
Shortest distance: 7  
Path: v3 -> v7  
  
--- Distance Vector from v3 ---  
v3: 0  
v4: 6  
v5: 7  
v6: 5  
v7: 7
```

Рис. 21: Построение сети потоков

После генерации сети пользователь может вывести матрицы пропускных способностей, стоимостей и текущего потока. Пример такого вывода показан на рис. 22.

```
===== MAIN MENU =====  
> 11  
  
--- Algorithm Speed Comparison ---  
Start vertex: 3  
  
=== Comparison Results ===  
BFS iterations:      27  
Dijkstra iterations: 50  
BFS is 1.85x times faster than Dijkstra.
```

Рис. 22: Матрицы пропускных способностей, стоимостей и потока

При выборе пункта максимального потока программа вычисляет значение потока и показывает результат работы алгоритма на текущей сети. Пример результата приведён на рис. 23.

```

===== MAIN MENU =====
> 12

[OK] Flow network built from the current graph.
Capacity(u, v) is generated randomly via the program generator
Cost(u, v) is generated randomly via the program generator
Vertices: 8
Edges: 25

```

Рис. 23: Вычисление максимального потока

Для потока минимальной стоимости программа использует найденный максимальный поток и строит поток величины $[2/3 \cdot max]$. Пример результата показан на рис. 24.

```

===== MAIN MENU =====
> 13

=== CAPACITY MATRIX ===
i = no directed edge

```

	0	1	2	3	4	5	6	7
0	i	4	i	3	6	7	4	i
1	i	i	3	5	5	5	4	4
2	i	i	i	i	8	7	5	7
3	i	i	i	i	7	6	5	5
4	i	i	i	i	i	7	4	3
5	i	i	i	i	i	i	6	3
6	i	i	i	i	i	i	i	5
7	i	i	i	i	i	i	i	i

Рис. 24: Вычисление потока минимальной стоимости

4.4. Результаты лабораторной работы №4

При выборе пункта теоремы Кирхгофа программа строит матрицу Лапласа и вычисляет число остовных деревьев текущего графа. Пример результата показан на рис. 25.

```

===== MAIN MENU =====
> 14

=== COST MATRIX ===
i = no directed edge

```

	0	1	2	3	4	5	6	7
0	i	6	i	5	3	4	6	i
1	i	i	8	7	5	2	4	5
2	i	i	i	i	2	6	6	4
3	i	i	i	i	4	6	6	6
4	i	i	i	i	i	5	7	6
5	i	i	i	i	i	i	6	3
6	i	i	i	i	i	i	i	4
7	i	i	i	i	i	i	i	i

Рис. 25: Вычисление числа остовных деревьев по теореме Кирхгофа

При построении минимального остова программа выводит рёбра, вошедшие в остов, и суммарный вес. Пример результата показан на рис. 26.

```

--- Maximum Flow (Ford-Fulkerson) ---
Source vertex: 3
Sink vertex: 6

[OK] Maximum flow = 15

Augmenting steps:
Step 1: v3 -> v6 | added flow = 5 | total = 5
Step 2: v3 -> v5 -> v6 | added flow = 6 | total = 11
Step 3: v3 -> v4 -> v6 | added flow = 4 | total = 15

Flow matrix after max flow:

```

	0	1	2	3	4	5	6	7
0	i	0	i	0	0	0	0	i
1	i	i	0	0	0	0	0	0
2	i	i	i	i	0	0	0	0
3	i	i	i	i	4	6	5	0
4	i	i	i	i	i	0	4	0
5	i	i	i	i	i	i	6	0
6	i	i	i	i	i	i	i	0
7	i	i	i	i	i	i	i	i

Рис. 26: Построение минимального остова алгоритмом Краскала

После этого можно отдельно вывести уже сохранённый остов. Пример такого вывода приведён на рис. 27.

```

===== MAIN MENU =====
> 16

--- Minimum-Cost Flow ---
Using the same source/sink as the maximum-flow run: v3 -> v6
Target flow = floor(2/3 * 15) = 10

=== Result ===
Target flow: 10
Achieved flow: 10
Total cost: 86
Success: yes

Augmenting steps:
Step 1: v3 -> v6
added flow: 5
path unit cost: 6
cumulative flow: 5
cumulative cost: 30
Step 2: v3 -> v4 -> v6
added flow: 4
path unit cost: 11
cumulative flow: 9
cumulative cost: 74
Step 3: v3 -> v5 -> v6
added flow: 1
path unit cost: 12
cumulative flow: 10
cumulative cost: 86

Flow matrix after minimum-cost flow:

```

	0	1	2	3	4	5	6	7
0	i	0	i	0	0	0	0	i
1	i	i	0	0	0	0	0	0
2	i	i	i	i	0	0	0	0
3	i	i	i	i	4	1	5	0
4	i	i	i	i	i	0	4	0
5	i	i	i	i	i	i	1	0
6	i	i	i	i	i	i	i	0
7	i	i	i	i	i	i	i	i

Рис. 27: Вывод построенного минимального остова

Для кодирования по Прюферу программа строит последовательность кода и сохраняет веса рёбер. Пример результата показан на рис. 28.

```
===== MAIN MENU =====
> 17

=== FLOW NETWORK ===
Type:      Directed
Vertices:  8
Edges:     25
Vertex IDs: 0 1 2 3 4 5 6 7

Edges:
v0 -> v5 | capacity = 7, cost = 4, flow = 0
v0 -> v3 | capacity = 3, cost = 5, flow = 0
v0 -> v1 | capacity = 4, cost = 6, flow = 0
v0 -> v4 | capacity = 6, cost = 3, flow = 0
v0 -> v6 | capacity = 4, cost = 6, flow = 0
v1 -> v2 | capacity = 3, cost = 8, flow = 0
v1 -> v4 | capacity = 5, cost = 5, flow = 0
v1 -> v5 | capacity = 5, cost = 2, flow = 0
v1 -> v6 | capacity = 4, cost = 4, flow = 0
v1 -> v7 | capacity = 4, cost = 5, flow = 0
v1 -> v3 | capacity = 5, cost = 7, flow = 0
v2 -> v6 | capacity = 5, cost = 6, flow = 0
v2 -> v5 | capacity = 7, cost = 6, flow = 0
v2 -> v7 | capacity = 7, cost = 4, flow = 0
v2 -> v4 | capacity = 8, cost = 2, flow = 0
v3 -> v6 | capacity = 5, cost = 6, flow = 5
v3 -> v7 | capacity = 5, cost = 6, flow = 0
v3 -> v5 | capacity = 6, cost = 6, flow = 1
v3 -> v4 | capacity = 7, cost = 4, flow = 4
v4 -> v5 | capacity = 7, cost = 5, flow = 0
v4 -> v6 | capacity = 4, cost = 7, flow = 4
v4 -> v7 | capacity = 3, cost = 6, flow = 0
v5 -> v7 | capacity = 3, cost = 3, flow = 0
v5 -> v6 | capacity = 6, cost = 6, flow = 1
v6 -> v7 | capacity = 5, cost = 4, flow = 0
```

Рис. 28: Кодирование остова кодом Прюфера

При декодировании программа восстанавливает дерево по ранее полученному коду Прюфера. Пример результата показан на рис. 29.

```

===== MAIN MENU =====
> 18

=== KIRCHHOFF MATRIX (B = D - A) ===
Diagonal: degree of vertex.
Off-diagonal: -1 if edge exists, 0 otherwise.

      0   1   2   3   4   5   6   7
-----
0 |   4  -1  -1   0  -1   0   0  -1
1 |  -1   4  -1  -1  -1   0   0   0
2 |  -1  -1   5  -1  -1   0  -1   0
3 |   0  -1  -1   5  -1  -1   0  -1
4 |  -1  -1  -1  -1   7  -1  -1  -1
5 |   0   0   0  -1  -1   4  -1  -1
6 |   0   0  -1   0  -1  -1   4  -1
7 |  -1   0   0  -1  -1  -1  -1   5

[OK] Number of spanning trees = 11999

```

Рис. 29: Декодирование дерева по коду Прюфера

Для поиска максимального независимого множества рёбер пользователь сначала выбирает, на каком графе запускать алгоритм: на исходном или на построенном остовет. Пример такого выбора показан на рис. 30, а примеры работы для двух случаев приведены на рис. 31 и 32.

```

===== MAIN MENU =====
> 18
[!] Lab 4 requires an undirected graph. Generate an undirected graph first.

```

Рис. 30: Выбор графа для запуска алгоритма паросочетания

```

===== MAIN MENU =====
> 19

=== KRUSKAL'S ALGORITHM ===
Edges added (in order, sorted by ascending weight):
 1. v0 --- v1 (weight = -9)
 2. v3 --- v4 (weight = -7)
 3. v5 --- v6 (weight = -7)
 4. v5 --- v7 (weight = -7)
 5. v0 --- v2 (weight = -5)
 6. v0 --- v7 (weight = -5)
 7. v4 --- v7 (weight = -5)

Total weight of MST: -45
Edges in MST:       7 of 7 expected

[OK] Spanning tree saved as the current MST.

```

Рис. 31: Поиск паросочетания на исходном графе

```

===== MAIN MENU =====
> 20

=== SPANNING TREE ===
Type:      Undirected
Vertices:  8
Edges:     7

Edges:
v0 --- v1 (weight = -9)
v0 --- v2 (weight = -5)
v0 --- v7 (weight = -5)
v3 --- v4 (weight = -7)
v4 --- v7 (weight = -5)
v5 --- v6 (weight = -7)
v5 --- v7 (weight = -7)

Total weight: -45

```

Рис. 32: Поиск паросочетания на построенном остове

4.5. Результаты лабораторной работы №5

При проверке эйлеровости программа анализирует граф и сообщает, является ли он эйлеровым или требует модификации. Пример начального результата показан на рис. 33.

```

===== MAIN MENU =====
> 21

=== PRUFER CODE ===
Tree has 8 vertices, code length = p - 1 = 7

Code:   [0, 0, 7, 4, 7, 5, 7]
Weights: [-9, -5, -5, -7, -5, -7, -7]

```

Рис. 33: Проверка графа на эйлеровость

Если граф не является эйлеровым, программа выполняет его модификацию и журналирует внесённые изменения. Пример такого этапа показан на рис. 34.

```

===== MAIN MENU =====
> 22

=== DECODED TREE ===
Vertices: 8
Edges:    7

Edges:
v0 --- v1 (weight = -9)
v0 --- v2 (weight = -5)
v0 --- v7 (weight = -5)
v3 --- v4 (weight = -7)
v4 --- v7 (weight = -5)
v5 --- v6 (weight = -7)
v5 --- v7 (weight = -7)

```

Рис. 34: Модификация графа при эйлеризации

После завершения эйлеризации программа выводит исходный граф и построенный остов, используемый далее для циклового анализа. Примеры этих двух представлений показаны на рис. 35 и 36.

```

===== MAIN MENU =====
> 23

--- Maximal Matching (independent edge set) ---
Run on:
  1. Original graph
  2. Spanning tree (option 19 must be done first)
> 1

Algorithm:
  1. Greedy (maximal by inclusion -- per lecture definition)
  2. Edmonds blossom (guaranteed MAXIMUM cardinality)
> 2

=== EDMONDS MAXIMUM MATCHING on original graph ===
Matching size: 4

Matching edges:
v0 --- v2 (weight = -5)
v1 --- v4 (weight = -4)
v3 --- v7 (weight = 4)
v5 --- v6 (weight = -7)

[OK] Matching is PERFECT -- every vertex is covered.

```

Рис. 35: Исходный граф перед анализом цикловой структуры

```

===== MAIN MENU =====
> 23

--- Maximal Matching (independent edge set) ---
Run on:
  1. Original graph
  2. Spanning tree (option 19 must be done first)
> 2

Algorithm:
  1. Greedy (maximal by inclusion -- per lecture definition)
  2. Edmonds blossom (guaranteed MAXIMUM cardinality)
> 2

=== EDMONDS MAXIMUM MATCHING on spanning tree ===
Matching size: 3

Matching edges:
v0 --- v1 (weight = -9)
v3 --- v4 (weight = -7)
v5 --- v6 (weight = -7)

Unmatched vertices: v2 v7

```

Рис. 36: Построенный остов для анализа цикловой структуры

Далее программа строит и выводит фундаментальную систему циклов. Пользователь видит список циклов, их хорды и последовательности вершин. Пример результата показан на рис. 37.

```

===== MAIN MENU =====
> 24

=== EULERIAN TRAVERSAL ===
[!] Cannot make this graph Eulerian without
    duplicating an existing edge -- the only
    possible eulerization turns it into a
    multigraph unless we remove some edges.
Choose how to continue:
d - try deleting existing edges
m - allow multigraph conversion
n - abort
> d
[1] Proceeding with edge-deletion eulerization.
[!] Graph was NOT Eulerian. Modifications applied:
    1. removed edge v2 --- v3 [fix odd parity by deleting edge]
    2. removed edge v4 --- v7 [fix odd parity by deleting edge]
Total modifications: 2

Eulerian cycle (17 edges, 18 vertex stops):
v0 -> v2 -> v1 -> v0 -> v4 -> v1 -> v3 -> v7 -> v5 -> v3 -> v4 -> v2 ->
v6 -> v4 -> v5 -> v6 -> v7 -> v0

```

Рис. 37: Фундаментальная система циклов

После выбора нескольких фундаментальных циклов программа выполняет их симметрическую разность и восстанавливает полученные циклы из итогового множества рёбер. Пример результата показан на рис. 38.

```

> 25

=== FUNDAMENTAL CYCLE SYSTEM ===
One fundamental cycle per chord (non-tree edge).
Total cycles: 12 (= |E| - |V| + 1 for a connected graph).

C1 (chord v0 --- v4):
Vertex sequence: v0 -> v7 -> v4 -> v0
Edges (3): (v0,v4), (v0,v7), (v4,v7)

C2 (chord v1 --- v2):
Vertex sequence: v1 -> v0 -> v2 -> v1
Edges (3): (v0,v1), (v0,v2), (v1,v2)

C3 (chord v1 --- v3):
Vertex sequence: v1 -> v0 -> v7 -> v4 -> v3 -> v1
Edges (5): (v0,v1), (v0,v7), (v1,v3), (v3,v4), (v4,v7)

C4 (chord v1 --- v4):
Vertex sequence: v1 -> v0 -> v7 -> v4 -> v1
Edges (4): (v0,v1), (v0,v7), (v1,v4), (v4,v7)

C5 (chord v2 --- v3):
Vertex sequence: v2 -> v0 -> v7 -> v4 -> v3 -> v2
Edges (5): (v0,v2), (v0,v7), (v2,v3), (v3,v4), (v4,v7)

C6 (chord v2 --- v4):
Vertex sequence: v2 -> v0 -> v7 -> v4 -> v2
Edges (4): (v0,v2), (v0,v7), (v2,v4), (v4,v7)

C7 (chord v2 --- v6):
Vertex sequence: v2 -> v0 -> v7 -> v5 -> v6 -> v2
Edges (5): (v0,v2), (v0,v7), (v2,v6), (v5,v6), (v5,v7)

C8 (chord v3 --- v5):
Vertex sequence: v3 -> v4 -> v7 -> v5 -> v3
Edges (4): (v3,v4), (v3,v5), (v4,v7), (v5,v7)

C9 (chord v3 --- v7):
Vertex sequence: v3 -> v4 -> v7 -> v3
Edges (3): (v3,v4), (v3,v7), (v4,v7)

C10 (chord v4 --- v5):
Vertex sequence: v4 -> v7 -> v5 -> v4
Edges (3): (v4,v5), (v4,v7), (v5,v7)

C11 (chord v4 --- v6):
Vertex sequence: v4 -> v7 -> v5 -> v6 -> v4
Edges (4): (v4,v6), (v4,v7), (v5,v6), (v5,v7)

C12 (chord v6 --- v7):
Vertex sequence: v6 -> v5 -> v7 -> v6
Edges (3): (v5,v6), (v5,v7), (v6,v7)

```

Рис. 38: Симметрическая разность выбранных фундаментальных циклов

Если в результате получается один цикл или несколько циклов, программа выводит соответствующее восстановление и завершает этап анализа. Пример итогового вывода показан на рис. 39.

```
===== MAIN MENU =====  
> 26  
  
--- Symmetric Difference of Cycles ---  
Available fundamental cycles: 1..12  
Enter cycle numbers separated by spaces and press Enter.  
> 1 3 2 6 6  
  
Result of C1 /_\ C3 /_\ C2 /_\ C6 /_\ C6:  
Edges (5): (v0,v2), (v0,v4), (v1,v2), (v1,v3), (v3,v4)  
Reconstructed cycle: v0 -> v2 -> v1 -> v3 -> v4 -> v0
```

Рис. 39: Восстановление цикла после симметрической разности

Заключение

В ходе выполнения лабораторных работ был создан единый программный комплекс по теории графов, объединяющий задачи генерации графов, их метрического анализа, обходов, кратчайших путей, сетей потоков, остовных деревьев, кодирования деревьев, паросочетаний и эйлеровых обходов.

Лабораторная работа №1

1. сформирован случайным образом связный ациклический ориентированный граф в соответствии с биномиальным распределением по степеням вершин;
2. реализовано использование ориентированного или неориентированного графа, полученного из сгенерированного ориентированного графа, в зависимости от дальнейшей задачи;
3. посчитаны эксцентриситеты вершин;
4. выделен центр графа;
5. определены диаметральные вершины графа;
6. реализован метод Шимбелла для сгенерированной весовой матрицы при заданном пользователем количестве рёбер;
7. реализована генерация весовой матрицы в соответствии с биномиальным распределением с поддержкой только положительных, только отрицательных и смешанных значений по выбору пользователя;
8. реализован вывод минимального и/или максимального пути методом Шимбелла в виде матрицы;
9. определена возможность построения маршрута от одной заданной вершины до другой;
10. реализовано определение количества маршрутов между двумя вершинами;
11. реализовано пользовательское меню программы.

Лабораторная работа №2

12. реализован обход вершин графа поиском в ширину;
13. сгенерирована весовая матрица с положительными или отрицательными весами по выбору пользователя;
14. найден кратчайший путь между двумя выбранными пользователем вершинами классическим алгоритмом Дейкстры;
15. реализован вывод не только расстояния, но и самого пути в виде последовательности вершин;
16. сравнены скорости работы реализованных алгоритмов по количеству итераций.

Лабораторная работа №3

17. на основе сгенерированного ориентированного графа случайным образом получена матрица пропускных способностей;
18. на основе сгенерированного ориентированного графа случайным образом получена матрица стоимости;
19. найден максимальный поток для полученного графа по алгоритму Форда–Фалкерсона;
20. вычислен поток минимальной стоимости величины $[2/3 \cdot max]$, где max — найденный максимальный поток;
21. для задачи потока минимальной стоимости использованы ранее реализованные алгоритмы поиска кратчайших путей.

Лабораторная работа №4

22. найдено число остовных деревьев заданного неориентированного графа с использованием матричной теоремы Кирхгофа;
23. построен минимальный по весу остов для сгенерированного неориентированного взвешенного графа с использованием алгоритма Краскала;

24. выполнено кодирование полученного остова с помощью кода Прюфера;
25. выполнено декодирование остова по коду Прюфера;
26. обеспечено сохранение весов рёбер при кодировании и декодировании;
27. реализован алгоритм нахождения максимального независимого множества рёбер на исходном графе;
28. реализован алгоритм нахождения максимального независимого множества рёбер на полученном остове;
29. обеспечен выбор пользователем графа, на котором запускается алгоритм паросочетания.

Лабораторная работа №5

30. проверено, является ли заданный неориентированный граф эйлеровым;
31. граф модифицирован при необходимости с логированием внесённых изменений;
32. построен эйлеров цикл;
33. построен кратчайший остов для заданного неориентированного графа алгоритмом из четвёртой лабораторной работы;
34. получена фундаментальная система циклов на основе построенного остова;
35. реализован вывод фундаментальной системы циклов на экран;
36. реализовано получение остальных циклов на основе фундаментальных с помощью операции симметрической разности;
37. обеспечен выбор пользователем циклов, участвующих в операции симметрической разности.

4.6. Преимущества программы

- программа объединяет алгоритмы всех пяти лабораторных работ в одном приложении, всё реализовано с одними и теми же типами данных (одномерные и двумерные списки целых чисел);
- модульная структура упрощает сопровождение кода и добавление новых алгоритмов;
- единое для всех лабораторных работ меню с возможностью пересоздания графа;
- возможность скрыть разделы в меню;
- программа выводит не только итоговые значения, но и промежуточные структуры, что удобно для дебаггинга.

4.7. Недостатки программы

- консольный формат делает работу с большими графами и матрицами менее удобной;
- не все использованные алгоритмы являются наиболее оптимальными;
- используются готовые типы данных, такие как `vector`; созданием своих типов данных с узким функционалом можно повысить производительность;

4.8. Возможности масштабирования

- консольный интерфейс может быть заменён или дополнен графической визуализацией — например, с помощью Qt;
- реализовать импорт и экспорт графов;
- реализовать сохранение логов;
- отдельные алгоритмы могут быть заменены на более эффективные версии без полной переработки всей программы;

- можно создать свои типы данных с узким функционалом, чтобы повысить производительность.

Список литературы

Список литературы

- [1] Edmonds J. Matching, Euler Tours and the Chinese Postman / Jack Edmonds, Ellis L. Johnson — Нидерланды: North-Holland Publishing Company, 1973. — 37 с.
- [2] Antii L. Competitive Programmer's Handbook / Antii Laaksonen — Helsinki: July 2018. — 296 с.
- [3] Kolmogorov V. Blossom V: A new implementation of a minimum cost perfect matching algorithm / Vladimir Kolmogorov — London — 15 с.
- [4] Р. Н. Вадзинский. Справочник по вероятностным распределениям / Р. Н. Вадзинский — Санкт-Петербург: Наука, 2001. — 296 с.
- [5] Секция «Телематика» [Электронный ресурс]. — URL: <https://tema.spbstu.ru/tgraph/> (дата обращения: 28.05.2026).
- [6] Хаггарти Р. Дискретная математика для программистов / Р. Хаггарти — Москва: ТЕХНОСФЕРА, 2012 — 400 с.